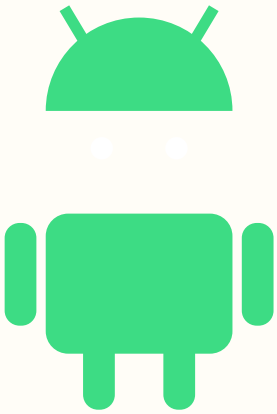
A brain-friendly guide for Android developers

Securing Your Flutter App

SSL pinning, obfuscation, secure storage, runtime protection and release hygiene - what to implement, why, and how, for Android developers.



what you know





what you'll learn

INSIDE:

- Think like an attacker: the threat model
- HTTPS, network security config & SSL pinning
- Obfuscation, R8 and keeping secrets out of the APK
- Secure storage, biometrics & backups
- Root/tamper detection, screenshots, logging
- Auth tokens, deep links, WebViews
- Supply chain + the release security checklist

Security isn't one lock on the front door. It's layers. Attackers only need ONE open window.

What's in this doc

1 **Think Like an Attacker**
Who wants into your app, and how

2 **Secure Networking & SSL Pinning**
Stop the man in the middle

3 **Protecting Your Code**
Obfuscation, R8 and the secrets you must not ship

4 **Protecting Data on the Device**
Secure storage, biometrics and backups

5 **Runtime Protection**
Root, tampering, screenshots and logs

6 **Auth, Deep Links & WebViews**
The doors users walk through

7 **Supply Chain & Release Checklist**
Your dependencies are your attack surface

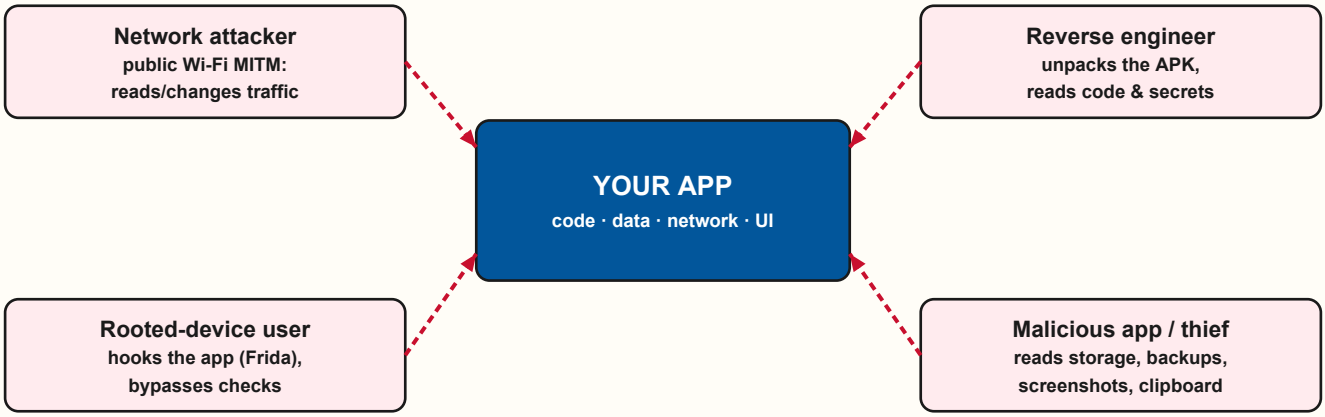
Everything in this doc applies to your native Android knowledge too. Flutter adds a few twists - they're marked with a big WATCH IT!

CHAPTER 1

Think Like an Attacker

Who wants into your app, and how

You can't defend against everyone and everything, so start by asking: **who** would attack this app, **what** do they want, and **where** can they get at it?



The OWASP MASVS map (your security syllabus)

The **OWASP Mobile Application Security Verification Standard** groups mobile risks into categories. Each chapter of this doc covers one or two of them.

Category	In plain English	Where
`MASVS-STORAGE`	Sensitive data stored securely on the device	Ch 4
`MASVS-CRYPTO`	Strong, correctly used cryptography; keys in Keystore	Ch 4
`MASVS-AUTH`	Secure authentication & session handling	Ch 6
`MASVS-NETWORK`	Secure communication (TLS, pinning)	Ch 2
`MASVS-PLATFORM`	Safe use of IPC, deep links, WebViews, UI	Ch 5, 6
`MASVS-CODE`	Up-to-date platform & dependencies, input validation	Ch 3, 7
`MASVS-RESILIENCE`	Resistance to reverse engineering & tampering	Ch 3, 5
`MASVS-PRIVACY`	Minimal data collection, transparency	Ch 7



BRAIN POWER

A to-do list app with no login, and a banking app. Which MASVS categories matter for each? Security effort should match the value of what you protect.

there are no **Dumb Questions**

Q: **Is a Flutter app harder to reverse engineer than a Kotlin app?**

A: A bit different, not "safe". Release Dart code is compiled to native ARM code in `libapp.so`, which is harder to read than DEX bytecode, but tools exist that recover class and function names and strings from it. Assume a determined attacker **will** read your code. Never rely on secrecy of the code for security.

Q: **So is client-side security pointless?**

A: No - it raises the cost of an attack and protects ordinary users from ordinary attacks (stolen phones, hostile Wi-Fi, malware). But the **server** must always enforce the real rules: authentication, authorization, validation, rate limits.

CHAPTER 2

Secure Networking & SSL Pinning

Stop the man in the middle

Layer 1: HTTPS everywhere (already done in this project)

android/app/src/main/AndroidManifest.xml + res/xml/network_security_config.xml

```
<application
    android:usesCleartextTraffic="false"          ← block http:// for Kotlin/Java
    android:networkSecurityConfig=
        "@xml/network_security_config"            ← res/xml/...
    android:allowBackup="false" ...>              ← see chapter 4

<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system" />               ← ignore user-installed CAs
    </trust-anchors>
  </base-config>
</network-security-config>
```

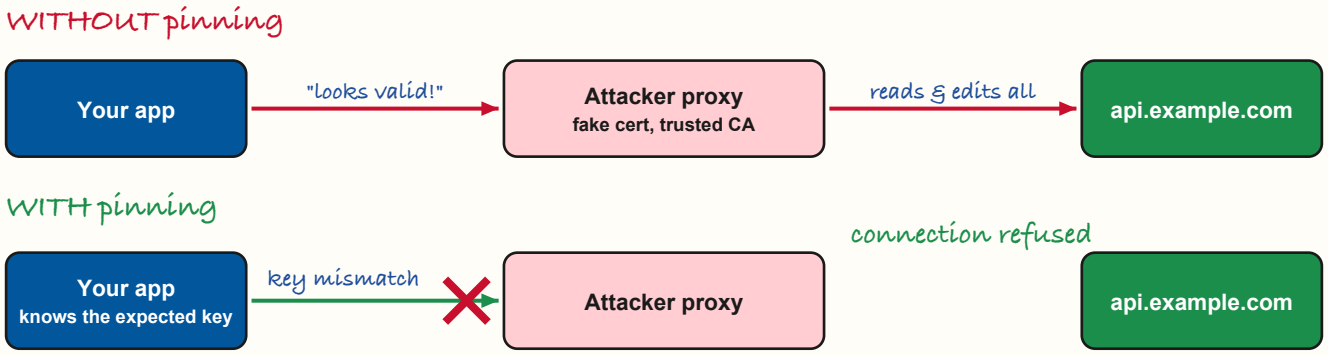


Watch it!

Flutter twist #1: Dart's `dart:io HttpClient` (used by `http` and `dio`) has its **own TLS stack**. Android's `network_security_config.xml` - including its `<pin-set>` - applies to Kotlin/Java networking and WebViews, **not** to your Dart HTTP calls. Keep the XML (it protects plugins and WebViews) but **implement pinning in Dart** as shown below. Also enforce `https://` URLs in Dart code yourself.

Layer 2: SSL (certificate) pinning

Normal TLS trusts *any* certificate signed by *any* of the ~150 trusted CAs on the phone. If an attacker gets one of those CAs to issue a cert for your domain - or tricks the user into installing their own CA - they can read and modify all your traffic. **Pinning** means your app accepts only the certificate (or public key) that you expect.



Pin to...	What you compare	Security	Maintenance
Leaf certificate	Exact server cert	Strongest	Breaks every renewal (often every 90 days!) - needs an app update
Leaf public key (SPKI)	Hash of the server's public key	Strong	Survives renewal if you reuse the key pair
Intermediate CA key	Hash of your CA's intermediate key	Good	Survives renewals; trusts any cert from that CA

Option A - trust only your own certificate (dart:io, no extra packages)

CERTIFICATE PINNING WITH SecurityContext

```
import 'dart:io';
import 'package:flutter/services.dart';
import 'package:http/io_client.dart';

Future<IOClient> pinnedClient() async {
  final pem = await rootBundle
      .load('assets/certs/api_ca.pem');
  final ctx = SecurityContext(withTrustedRoots: false)
    ..setTrustedCertificatesBytes(
        pem.buffer.asUint8List());
  final http = HttpClient(context: ctx)
    ..badCertificateCallback = (_, __, ___) => false;
  return IOClient(http);
}
```

← your CA/intermediate cert
← ignore ALL system CAs

← trust only this one

← never "accept anyway"
← drop-in http.Client

Option B - public-key / fingerprint pinning with dio

FINGERPRINT PINNING WITH dio 5

```
import 'dart:io';
import 'package:crypto/crypto.dart';
import 'package:dio/dio.dart';
import 'package:dio/io.dart';

const _pins = {
  'a1b2c3...current-cert-sha256',
  'd4e5f6...next-cert-sha256',
};

Dio pinnedDio() => Dio()
  ..httpClientAdapter = IOHttpClientAdapter(
    validateCertificate: (cert, host, port) {
      if (cert == null) return false;
      final fp = sha256.convert(cert.der).toString();
      return _pins.contains(fp);
    },
  );
```

← sha256

← current + BACKUP pin

← rotate without an outage

← runs on EVERY connection

← fingerprint of the leaf

← false = abort request

TERMINAL

```
# Get the SHA-256 fingerprint of your server's current certificate (DER):
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null \
  | openssl x509 -outform DER | openssl dgst -sha256
```

Prefer a packaged solution? `http_certificate_pinning` and similar plugins check SPKI hashes natively. Whatever you choose, test it: point the app at a proxy like mitmproxy/Burp with its CA installed - requests **must fail**.



Watch it!

Pinning can **brick your app**. If the server certificate changes and no shipped pin matches, every user loses connectivity until they update. Always ship a **backup pin**, coordinate certificate rotation with your backend team, track pin expiry dates, and consider a remote kill-switch for pins (fetched over a separately pinned or signed channel).

there are no Dumb Questions

Q: **Why doesn't Burp see my Flutter app's traffic even without pinning?**

A: dart:io ignores the Android system proxy setting. That's not a security feature to rely on - attackers with root can redirect traffic anyway (and tools exist to patch Flutter apps). It just surprises developers during debugging.

Q: **Is pinning enough on its own?**

A: No. On a rooted device an attacker can hook the check and return "true". Pinning protects honest users on hostile networks; combine it with server-side authorization and the runtime protections in Chapter 5 for high-value apps.

CHAPTER 3

Protecting Your Code

Obfuscation, R8, and the secrets you must not ship

An APK is a zip file. Anyone can download it and look inside. Here's what they find:

WHAT AN ATTACKER SEES

```
app-release.apk
├── AndroidManifest.xml          ← permissions, components
├── classes.dex                  ← Kotlin/Java + plugin code (R8)
├── lib/arm64-v8a/
│   ├── libflutter.so            ← the Flutter engine
│   └── libapp.so                ← YOUR Dart code, AOT-compiled
├── assets/flutter_assets/       ← images, fonts, JSON, .env files!
└── res/                         ← Android resources
```

Dart obfuscation

TERMINAL - always use both flags together

```
flutter build appbundle --release \
    --obfuscate \
    --split-debug-info=build/symbols/1.4.2
```

What obfuscation does	Example	Protects?
Renames classes, methods, fields	PaymentService.verifyPin → a.b	Yes
Removes debug symbols from the binary	Smaller app, fewer hints	Yes
Hides string literals	'https://api...' and 'API_KEY' stay readable	NO
Hides logic	Control flow can still be analysed	NO
Encrypts assets	flutter_assets/ is plain files	NO

R8 for the native side

Flutter release builds already run R8 on the Kotlin/Java side. This project states it explicitly (`isMinifyEnabled = true`, `isShrinkResources = true`) and adds a `proguard-rules.pro` that strips `Log.d/v/i` calls. If a plugin uses reflection and breaks in release, add its keep rules there.

The golden rule of secrets

If it ships in the app, it is PUBLIC. Obfuscation only slows people down.

Where the secret lives	Reality	Verdict
Hard-coded in Dart: const apiSecret = 'sk_live_...'	Extractable (strings are not obfuscated)	Never
--dart-define=API_KEY=...	Compiled into libapp.so - extractable	Only for non-secret config
.env file in assets (flutter_dotenv)	Plain file inside the APK	Only for non-secret config
google-services.json API key	Public by design; restrict it in Cloud Console	OK if restricted
Your backend holds the secret; app calls backend	Secret never leaves the server	YES - do this
Short-lived, per-user tokens from your backend	Limited blast radius if stolen	YES
Firebase App Check / Play Integrity on the backend	Backend only answers genuine app instances	YES, as extra layer



Watch it!

Keep your `build/symbols` folder **private** (CI artifacts, not a public repo). It contains the mapping that undoes obfuscation - like `mapping.txt` for R8.

CHAPTER 4

Protecting Data on the Device

Secure storage, biometrics and backups

Data	Use	Why
Settings, theme, onboarding done	shared_preferences	Plain XML file - fine for non-sensitive data
Access / refresh tokens, PINs, keys	flutter_secure_storage	Encrypted; keys held in Android Keystore / iOS Keychain
Large sensitive datasets	drift / sqflite + SQLCipher (sqlcipher_flutter_libs)	Whole database encrypted; key in secure storage
Files (PDFs, images) with personal data	path_provider app directory + encryption (e.g. package:cryptography)	Never external/shared storage
Passwords the user types	Don't store them	Exchange for a token at login

ANDROID (Kotlin)

```
// Android: EncryptedSharedPreferences /
// Keystore-backed encryption
val prefs = EncryptedSharedPreferences.create(
    context, "secure",
    masterKey,
    AES256_SIV, AES256_GCM)
prefs.edit()
    .putString("token", token)
    .apply()
```

FLUTTER (Dart)

```
// Flutter: flutter_secure_storage
const storage = FlutterSecureStorage();

await storage.write(
    key: 'access_token', value: token);
final token =
    await storage.read(key: 'access_token');

// on logout
await storage.deleteAll();
```



Watch it!

This project's **Storage** demo uses `shared_preferences` on purpose (it's a learning demo of SharedPreferences). On a rooted phone that file is readable plain text: `/data/data/<app id>/shared_prefs/FlutterSharedPreferences.xml`. Never put tokens there.

Biometric unlock

LOCAL AUTHENTICATION

```
final auth = LocalAuthentication();
final canCheck = await auth.canCheckBiometrics
    || await auth.isDeviceSupported();
if (canCheck) {
  final ok = await auth.authenticate(
    localizedReason: 'Unlock your wallet',
  );
  if (ok) showVault();
}
```

← package:local_auth

← ≈ BiometricPrompt

Biometrics only gate the *UI* unless the underlying key requires user authentication. For real protection, tie decryption keys to authentication in the Keystore (via a plugin option or a small platform channel).

Backups, clipboard and keyboards

DATA-AT-REST CHECKLIST

- ☐ `android:allowBackup="false"` (done in this project), or use `dataExtractionRules` to exclude sensitive files from cloud backup and device transfer.
- ☐ For secret fields: `obscureText: true, enableSuggestions: false, autocorrect: false` - keeps passwords out of keyboard learning dictionaries.
- ☐ Avoid putting OTPs, tokens or card numbers on the clipboard; if you must, clear it after a short delay.
- ☐ Clear caches, secure storage and in-memory state on logout.
- ☐ Minimise what you store at all: data you don't keep can't leak.

CHAPTER 5

Runtime Protection

Root, tampering, screenshots and logs

Root / jailbreak, hooking & tamper detection

Tool	What it tells you	Use it for
Play Integrity API	Google attests the app binary is genuine, installed from Play, on a genuine device. Verify the token ON YOUR SERVER.	Server-side decisions; strongest signal
freeRASP (freerasp)	On-device checks: root, emulator, debugger, hooking (Frida), repackaging/signature, untrusted installer.	React in-app: warn, limit features, report
Firebase App Check	Uses Play Integrity / DeviceCheck so only your genuine app can call your Firebase / backend APIs.	Protects APIs from scripts



RELAX

Don't crash the app when you detect root - many legitimate users root their phones. Decide per feature: a game might just log it; a bank might block payments but allow viewing the balance. And remember every on-device check can eventually be bypassed; server-side attestation is harder to fake.

Stop screenshots & hide content in Recents

ANDROID (Kotlin)

```
// MainActivity.kt - via a MethodChannel
"setSecure" -> {
  val on = call.arguments as Boolean
  if (on) window.addFlags(
      WindowManager.LayoutParams.FLAG_SECURE)
  else window.clearFlags(
      WindowManager.LayoutParams.FLAG_SECURE)
  result.success(null)
}
```

FLUTTER (Dart)

```
// Dart - on a sensitive screen
@override
void initState() {
  super.initState();
  _channel.invokeMethod('setSecure', true);
}

@override
void dispose() {
  _channel.invokeMethod('setSecure', false);
  super.dispose();
}
```

FLAG_SECURE blocks screenshots, screen recording and shows a blank thumbnail in Recents. (Packages such as `screen_protector` wrap this for you.) You can also react to `AppLifecycleState.inactive` by showing a blur overlay - see Doc 3, Chapter 3.

Logging: the leak nobody notices



Watch it!

Flutter twist #2: `print()` and `debugPrint()` **still print in release builds** and show up in `adb logcat`. Never log tokens, passwords, personal data or full API responses. Guard debug logging with `kDebugMode` or use the `logging` package with a release level of OFF.

SAFE DEBUG LOGGING

```dart
import 'package:flutter/foundation.dart';

void logDebug(String msg) {
  if (kDebugMode) debugPrint(msg);
}

logDebug('Token: $token');
```

← compiled out of release builds

← still: don't log secrets at all!

Error messages & debug features



BULLET POINTS

- Show friendly error messages to users; send details to Crashlytics/Sentry, not to the screen.
- Remove or guard debug menus, test endpoints and "skip login" buttons with kDebugMode - release builds strip code inside `if (kDebugMode)`.
- Keep `debugShowCheckedModeBanner: false` for screenshots - but note that release builds never show it anyway.

CHAPTER 6

Auth, Deep Links & WebViews

The doors users walk through

Authentication done right



OAuth 2.0 Authorization Code + PKCE (flutter_appauth)

AUTH CHECKLIST

- ☐ Use OAuth 2.0 / OpenID Connect with **PKCE** through the system browser (flutter_appauth) - never collect third-party passwords in your own WebView.
- ☐ Store tokens only in **flutter_secure_storage**. Keep access tokens short-lived; rotate refresh tokens.
- ☐ Send tokens only over HTTPS (pinned), in the Authorization header - never in URLs (they end up in logs).
- ☐ On logout: revoke on the server, delete from secure storage, clear caches.
- ☐ The **server** checks every request: is this token valid, and is this user allowed to do this?
- ☐ Rate-limit login and OTP endpoints on the server.

Deep links: treat incoming data as hostile

ANDROID (XML)

```
<!-- AndroidManifest.xml: verified App Link -->
<intent-filter android:autoVerify="true">
  <action android:name=
      "android.intent.action.VIEW"/>
  <category android:name=
      "android.intent.category.DEFAULT"/>
  <category android:name=
      "android.intent.category.BROWSABLE"/>
  <data android:scheme="https"
        android:host="app.example.com"/>
</intent-filter>
<!-- + /.well-known/assetlinks.json
     on your server -->
```

FLUTTER (Dart)

```
// go_router: validate before acting
GoRoute(
  path: '/order/:id',
  redirect: (context, state) {
    final id = state.pathParameters['id']!;
    final valid =
        RegExp(r'^[0-9]{1,10}$').hasMatch(id);
    if (!valid) return '/';
    if (!auth.isLoggedIn) return '/login';
    return null;          // allowed
  },
  builder: (_, state) => OrderScreen(
      id: state.pathParameters['id']!),
)
```

Prefer verified **https App Links** over custom schemes (`myapp://`), which any app can register. Never let a deep link trigger a payment, a transfer or a settings change without confirmation and authentication.

WebViews (webview_flutter)

Setting	Secure choice
JavaScript	Leave JavaScriptMode.disabled unless the page needs it
Navigation	Use NavigationDelegate(onNavigationRequest:) to allow only your domains
JavaScript channels	addJavaScriptChannel only for pages you fully control; validate every message
Content	Load https:// only; never load untrusted HTML strings
Login pages	Don't - use the system browser (Custom Tabs) via flutter_appauth / url_launcher

Permissions & platform



BULLET POINTS

- Request the **minimum** permissions, at the moment you need them (permission_handler), with an explanation.
- Keep exported components to a minimum. FlutterActivity is exported because it's the launcher; anything you add should be `android:exported="false"` unless it must be public.
- Validate all data arriving through platform channels on the Kotlin side too.

CHAPTER 7

Supply Chain & Release Checklist

Your dependencies are your attack surface

Every package in `pubspec.yaml` runs with your app's full permissions - including its native Android code. Choose them like you'd choose a new team member.

Check	How
Verified publisher	Shield icon on pub.dev (e.g. dart.dev, flutter.dev, firebase.google.com)
Pub points & popularity	High pub points = documented, analysed, maintained; likes/downloads show adoption
Maintenance	Recent releases, responsive issue tracker, supports current Flutter/Dart
Permissions & native code	Read its AndroidManifest and Kotlin code - does it need what it asks for?
Size of the dependency tree	dart pub deps - fewer transitive packages, smaller attack surface

TERMINAL

```
flutter pub outdated          # what's behind
flutter pub upgrade --major-versions   # (review changelogs first!)
dart pub deps                 # full dependency tree
osv-scanner --lockfile=pubspec.lock    # known vulnerabilities (Google OSV database)
```



BULLET POINTS

- Commit `pubspec.lock` so builds are reproducible.
- Enable Dependabot/Renovate for pub dependencies and review updates like code.
- Keep Flutter itself up to date - engine updates include security fixes (BoringSSL, Skia...).
- Keep the keystore, key.properties and symbols out of git; use CI secrets.
- Privacy: declare everything in Play's Data safety form, including what SDKs collect.

Spot the vulnerabilities!



SHARPEN YOUR PENCIL

This code "works". Find at least **six** security problems before peeking at the solutions.

lib/auth.dart (DON'T COPY THIS)

```
const apiKey = 'sk_live_51H...';

Future<void> login(String user, String pass) async {
  final res = await http.get(Uri.parse(
    'http://api.example.com/login?u=$user&p=$pass&key=$apiKey'));
  print('Login response: ${res.body}');
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('token', jsonDecode(res.body)['token']);
  await prefs.setString('password', pass);
}
```

THE RELEASE SECURITY CHECKLIST

- ☐ **Network:** HTTPS only; cleartext disabled; Dart-side pinning with a backup pin; tested against a proxy.
- ☐ **Code:** built with --obfuscate --split-debug-info; symbols archived privately; R8 enabled.
- ☐ **Secrets:** no API secrets in Dart, --dart-define or assets; backend proxies secret calls.
- ☐ **Storage:** tokens in flutter_secure_storage; sensitive DB encrypted; allowBackup / extraction rules set.
- ☐ **Auth:** OAuth + PKCE via system browser; short-lived tokens; logout clears everything.
- ☐ **Runtime:** Play Integrity / App Check verified server-side; FLAG_SECURE on sensitive screens.
- ☐ **Logging:** no print/debugPrint of personal data or tokens; debug logs behind kDebugMode.
- ☐ **Platform:** minimal permissions; verified App Links; deep-link parameters validated; WebViews locked down.
- ☐ **Dependencies:** reviewed packages; lock file committed; vulnerability scan in CI; Flutter up to date.
- ☐ **Release:** signed with the upload key; keystore backed up and not in git; Data safety form accurate.



BULLET POINTS

- Assume attackers can read your app. The server enforces the real rules.
- Android's network_security_config doesn't cover Dart HTTP - pin in Dart, with a backup pin.
- Obfuscation renames symbols; it doesn't hide strings or secrets.
- Tokens go in flutter_secure_storage, never SharedPreferences.
- print/debugPrint leak into logcat in release builds.
- Layer defences: TLS + pinning + secure storage + attestation + server checks.

Exercise Solutions

Chapter 1 - to-do app vs banking app

To-do app (no login): mostly STORAGE basics, CODE (updated dependencies) and PRIVACY. Banking app: every category, with emphasis on NETWORK (pinning), AUTH, STORAGE, RESILIENCE (integrity checks) and PLATFORM (FLAG_SECURE, deep links).

Chapter 7 - spot the vulnerabilities

#	Problem	Fix
1	Secret API key hard-coded in Dart	Strings survive obfuscation. Move the secret to your backend.
2	http:// instead of https://	Credentials travel in clear text. Use HTTPS (+ pinning).
3	Credentials in the URL query string (GET)	URLs get logged by proxies/servers. POST them in the body over TLS.
4	print() of the full response	Leaks the token to logcat, even in release.
5	Token in SharedPreferences	Plain-text XML. Use flutter_secure_storage.
6	Storing the password at all	Never store passwords; keep only a (refreshable) token.
7	No status code / error handling	Crashes or stores garbage on failure; check res.statusCode.
8	Unvalidated input interpolated into the URL	Encode parameters (Uri(queryParameters:)) - or better, a JSON body.